Agent Development Kit (ADK) by Michaël Bettan

Definition: ADK is Google's open-source, code-first framework (Python, TypeScript, Java, Go, Kotlin) for building, evaluating and deploying AI agents. It supplies the agent loop as infrastructure: an execution engine, tool runtime, scoped state and memory, multi-agent delegation, guardrail hooks, evaluation and one-command deployment.

Why ADK? (Raw model API limits): a model API is **stateless** (remembers nothing between calls), **cannot act** (it can only *request* a tool call), has **no control flow** (no durable loop, branching, checkpointing or stop condition) and **no governance** (no audit trail, approval gate or isolation between users). ADK supplies all four, so you build the agent instead of the plumbing.

> MENTAL MODEL
> The model is the **brain**. ADK is the **body and nervous system**: hands (tools), memory (sessions), reflexes (callbacks) and a spine (the Runner) that decides when the brain thinks and when the hands move.

Workflow: 1. Define (`Agent` + tools + instruction) → **2. Run** (`Runner` loads the session, calls the model, executes tools, commits every step as an `Event`) → **3. Evaluate** (`adk eval` grades answers *and* tool trajectories) → **4. Deploy** (Agent Runtime, Cloud Run or GKE).

1. When to use ADK (vs. a single call, a workflow or another framework)

> RULE 0
> Start with a single LLM call. Add workflows, then agents, **only when evals prove the simpler version fails.**

Layer	What it is	Decides?	Acts?
LLM	Stateless next-token predictor	✅	❌ (can only request a tool call)
AI agent	LLM in a loop that executes tools, observes results and persists state until a goal is met	✅	✅
ADK	Framework that supplies that loop, plus state, guardrails, evals and deployment, as production infrastructure	n/a	n/a

Situation	Best fit	Why
One prompt (+ retrieval) already gives the answer	Plain / augmented LLM call	An agent only adds latency, cost and variance
Fixed, known steps (e.g. a 5-step contract review)	**Workflow** (`Workflow` graph in ADK 2.0, or `SequentialAgent` / `ParallelAgent` / `LoopAgent`)	Deterministic routing, predictable cost
Q&A over enterprise documents	RAG pipeline (see the RAG guide)	No decision loop needed
Next step depends on what the last step discovered	**`LlmAgent`** (ReAct loop)	Path can't be drawn in advance
Regulated, must be reproducible	Deterministic code / classic ML	Auditable, no model variance

Workflows vs. agents: a *workflow* orchestrates LLMs and tools through **predefined code paths** (railway tracks). An *agent* **directs its own process** (a taxi). Both are "agentic systems".

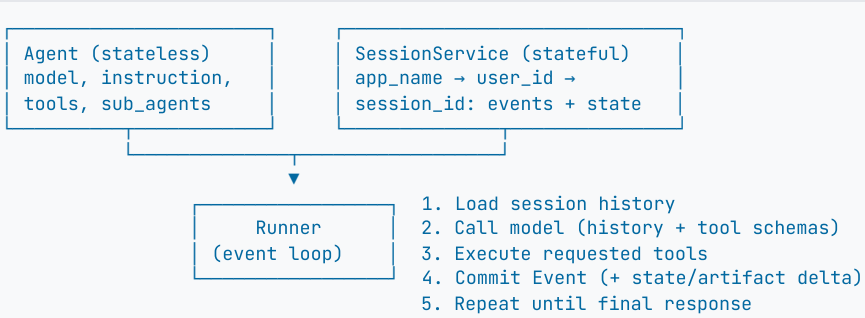
The "agent theater" litmus test: multi-agent is justified **only if sub-agents have independent tools, private data, an independent verifier role (`include_contents='none'` + external check) or human-gated steps.** Splitting one prompt into three persona agents plus a synthesizer costs ~4× the tokens and 1.5–2.5× the latency for marginal quality gains.

Framework	Mental model	Choose when
ADK	`Agent` + `Runner` + `Session Service` , workflow agents, graph `Workflow`	Google Cloud / Gemini, explicit Python control flow, managed sessions and Memory Bank
LangGraph	`StateGraph` with nodes, conditional edges, checkpointer	Complex cyclic graphs, time-travel debugging, `interrupt()` pauses
CrewAI	Role/goal/backstory teams	Fast prototypes of conversational teams
PydanticAI	Type-safe agent functions	Schema-first API backends

> COMMON TRAP
> "SDK" ≠ "ADK". The Gen AI SDK (`google-genai`) is a model API client: it can run a short in-process function-calling helper (10 calls default) and keep in-memory chat history, but has no persistent sessions, event ledger, multi-agent delegation, callbacks/plugins, evals or deployment. ADK is the agent framework built on top of it.

2. Runtime architecture: Agent, Runner and SessionService

ADK separates the **brain**, the **memory** and the **engine**:



Primitive	Role	Key rule
`Agent` (`Llm` `Agent`)	Spec: model, instruction, tools, sub-agents, callbacks	Holds **no** conversation state, so one instance safely serves all users
`BaseLlm`	Model adapter (`model="gemini-..."`, `LiteLlm("openai/...")`, `Claude(...)`, `Gemma(...)`)	Swap models per agent without changing tools, sessions or callbacks
`Runner`	Drives turns via `run_async(...)` or live audio/video via `run_live(..., live_request_queue=...)`	**Commit-then-resume:** each event is persisted before the agent continues
`Session Service`	Stores events and state per `app_name` / `user_id` / `session_id`	`InMemorySessionService` (dev) → `DatabaseSessionService` (`sqlite+aiosqlite` / `postgresql+asyncpg`) / `VertexAi SessionService` (prod; `app_name` = Reasoning Engine ID); note: tool confirmation is `InMemory`-only
`Memory Service`	Cross-session recall	`InMemoryMemoryService` (tests) → `VertexAiMemoryBankService` / `VertexAiRagMemoryService`
`Artifact Service`	Binary files and generated assets (versioned; `"user:..."` filename persists across sessions)	`InMemoryArtifactService` / `FileArtifactService` (dev) → `GcsArtifactService` (prod)
`App`	Top-level container for `root_agent`, `plugins=[...]` and app-wide configs	Home of `ResumabilityConfig`, `EventsCompactionConfig`, `ContextCacheConfig`

Context objects: `InvocationContext` (one per turn: session, agent, run config), `ReadonlyContext` (`InstructionProvider`: read-only `ctx.state`), `CallbackContext` (agent/model callbacks: state, artifacts), `ToolContext` (tools **and** tool callbacks: state, artifacts, auth, `search_memory`, confirmation, `actions`).

Event ledger: every step is appended to `session.events` as an `Event` with `author`, `invocation_id`, `content` (text or function call/response parts) and `actions` (`state_delta`, `artifact_delta`, `transfer_to_agent`, `escalate`, `skip_summarization`). One ledger feeds debugging (`adk web`), audit trails, evaluation (tool trajectories) and memory extraction.

> **COMMON TRAP**
> putting memory *inside* the agent object (`Agent(memory=Buffer())`). Under concurrent requests users' data bleeds together. State belongs to the `SessionService`, partitioned by the **3-tier isolation keys** `app_name` → `user_id` → `session_id`.

3. Agent types and deterministic control flow

All agents derive from `BaseAgent`. Only `LlmAgent` uses a model to decide; **workflow agents never call a model to route.** (In ADK 2.0, `SequentialAgent`, `ParallelAgent`, `LoopAgent` and custom `BaseAgent` workflows are **deprecated in favor of graph `Workflow`** in §5, though both appear on exams.)

Agent	Control flow	Use for	Key parameter
`LlmAgent` (alias `Agent`)	Model-driven (ReAct)	Reasoning, tool use, dynamic routing	`output_key`, `output_schema`, `sub_agents`, `tools`, `planner`
`Sequential Agent`	Fixed order 1 → 2 → 3	Pipelines where each step feeds the next	`sub_agents`
`Parallel Agent`	Concurrent fan-out (isolated `branch` tag, shared `session.state`)	Independent retrievals or checks	Distinct `output_key` per branch
`LoopAgent`	Repeat until exit	Refinement, polling, critic loops	`max_iterations` + `escalate`
Custom (`Base Agent`)	Your Python	Conditional branching, state machines	Override `_run_async_impl(ctx)`

```
from google.adk.agents import LlmAgent, SequentialAgent, ParallelAgent,
LoopAgent
from google.adk.tools import ToolContext

def exit_loop(tool_context: ToolContext) -> dict:
    """Call ONLY when the critique says the draft needs no further
changes."""
    tool_context.actions.escalate = True           # stops the
enclosing LoopAgent
    tool_context.actions.skip_summarization = True # skip extra LLM
call after exit tool
    tool_context.state["temp:approved"] = True     # distinguish
approval from max_iterations
    return {"status": "approved"}

research = ParallelAgent(name="research", sub_agents=[
    LlmAgent(name="market", instruction="Find market data.", tools=
[search], output_key="market"),
    LlmAgent(name="pricing", instruction="Find competitor pricing.",
tools=[search], output_key="pricing"),
])
refine = LoopAgent(name="refine", max_iterations=3, sub_agents=[
    LlmAgent(name="writer", instruction="Write or revise using
{market}, {pricing}, {critique?}.", output_key="draft"),
    LlmAgent(name="critic", instruction="Critique {draft}. If no issues
remain, call exit_loop.",
             tools=[exit_loop], output_key="critique"),
])
root_agent = SequentialAgent(name="report", sub_agents=[research,
refine])
```

- **Data passing:** `output_key="x"` saves the agent's final response to `state["x"]`; downstream instructions read it as `{x}` (`{x?}` if it may be missing; `{artifact.name}` injects a text artifact).
- **Dynamic instructions (`InstructionProvider`):** if an instruction contains literal curly braces (JSON examples, regexes, format strings) or needs branching logic, pass a callable `def build_prompt(ctx: ReadonlyContext) -> str:`: ADK skips `{var}` auto-templating when `instruction` is a function.
- **Loop termination:** a `LoopAgent` stops when a sub-agent or tool sets `escalate=True`, or when `max_iterations` trips. **When `max_iterations` trips, `LoopAgent` exits silently** (no exception), and the next step in an outer `SequentialAgent` still runs: record `temp:approved = True` in the exit tool and gate the downstream step via `before_agent_callback`.
- **Reflection cost:** every critic cycle adds model calls and grows history. To prevent self-grading bias, use a separate verifier (`temperature=0`, `include_contents='none'`, its own verification tools, explicit pass rubric).

> **COMMON TRAP**
> a `SequentialAgent` does **not** route; it runs every sub-agent in order. A `LoopAgent(max_iterations=3)` around "write code → run tests" runs all 3 iterations even when the tests pass, unless something sets `escalate`.

4. Multi-agent delegation: transfer, AgentTool and remote agents

Mechanism	How it works	Who answers the user next	Context & state	Use for
`sub_agents` (LLM transfer)	ADK injects `transfer_to_agent(agent_name=...)`; the model picks a target by each sub-agent's **`description`**	The **child** (it takes over)	Shared session history and state	Routing a whole request to a specialist (triage)
`AgentTool(agent=...)`	Parent calls the child like a function (`skip_summarization=True` returns raw output)	The **parent** (result returns as a tool response)	Child runs in a fresh child session, **and forwards its state/artifact deltas** (`output_key`) back to the parent	One sub-result, then continue
`RemoteA2aAgent`	Wraps a remote agent from its Agent Card over A2A	Depends on whether it's used as transfer target or tool	Opaque: only messages and artifacts cross	Cross-team or cross-org delegation

- **Transfer controls:** `disallow_transfer_to_parent=True` (blocks upward transfer **and** routes the next user turn back to the root agent: use on one-shot utility sub-agents, leave `False` on multi-turn specialists) and `disallow_transfer_to_peers=True` (forces hub-and-spoke through the coordinator).
- **Single-parent rule:** an agent instance can be added to `sub_agents` **only once** (`ValueError` if shared across two parents); use factory functions or `agent.clone()` to reuse a specialist definition.
- **ADK 2.0 sub-agent modes:** `mode="chat"` (default: one `transfer_to_agent` tool, child takes over conversation), `mode="task"` (child can ask the user clarifying questions, then returns structured results to the parent via `finish_task`), `mode="single_turn"` (child runs autonomously with no user interaction, can execute in parallel, and injects a delegation tool named after each sub-agent).
- **Cheaper routing:** when categories are stable, a custom `BaseAgent` router (rules or embedding similarity) avoids an LLM routing call; use a small model (Flash-Lite) when an LLM router is needed.
- **Handoff vs. graph:** use **handoffs** when one specialist fully owns a request and failures stay contained. Use a **graph / workflow** when steps depend on each other, several agents co-produce one output or a human must sign off.

Topology	Shape	Trade-off
Coordinator	`LlmAgent(sub_agents=[...])` routes requests	Extra routing call per request
Hierarchical	Nested coordinators decompose ambiguous goals	Best quality, highest latency and cost
Swarm / peer transfer	Specialists hand off without a central coordinator	Hardest to debug; needs an explicit exit
Generator–auditor	Generator drafts; a separate auditor (`temperature=0`, `include_contents='none'`, its own tools) verifies in a capped loop	Avoids self-grading bias; one extra call per round

> **COMMON TRAP**
> in ADK 1.x / `mode="chat"`, per-agent tools like `transfer_to_billing` do not exist: ADK uses **one** auto-injected `transfer_to_agent(agent_name=...)` tool (only ADK 2.0 `mode="single_turn"` / `"task"` generates per-agent delegation tools).

5. Graph workflows (ADK 2.0), planners and custom agents

ADK 2.0 replaces `SequentialAgent` / `ParallelAgent` / `LoopAgent` with a unified graph engine: `Workflow(edges=[...])` wires `LlmAgent`s, plain Python functions (`FunctionNode`) and nested `Workflow`s into a directed graph. Function return values flow to the next node as typed input (`node_input`); parameters matching state keys are injected automatically.

```
from google.adk import Workflow
from google.adk.workflow import START, END, JoinNode
from google.adk.events import Event

def route_ticket(node_input: str) -> Event:        # conditional edge via Event(route=...)
    target = "urgent_agent" if "outage" in node_input.lower() else "standard_agent"
    return Event(output=node_input, route=target)

root_agent = Workflow(name="triage", edges=[
    (START, route_ticket, ("urgent_agent", "standard_agent")),
    (("urgent_agent", "standard_agent"), END),
])
```

Concept	LangGraph	ADK 2.0 `Workflow` / ADK 1.x
Graph & branching	`StateGraph` + `add_conditional_edges`	`Workflow(edges=[...])` + `Event(route="target")` (or custom `BaseAgent`)
Fan-out / fan-in	Parallel edges + state reducer (`operator.add`)	Tuple fan-out `(a, (b, c))` + **`Join Node`** (merges into `dict[node_id, output]`); **no state reducers**: use distinct keys
Persistence	Checkpointer keyed by `thread_id`	`SessionService` keyed by `app_name` / `user_id` / `session_id`
Pause for a human	`interrupt()`	`RequestInput(message=..., schema=...)` yielded in a generator node, or tool confirmation / `LongRunningFunctionTool`

- **Custom agent (ADK 1.x):** subclass `BaseAgent`, override `async def _run_async_impl(self, ctx)` and `async for event in child.run_async(ctx): yield event`.
- **Planners on `LlmAgent(planner=...)`:** `BuiltInPlanner(thinking_config=types.ThinkingConfig(include_thoughts=True, thinking_budget=...))` enables Gemini native thinking tokens (`part.thought == True`); `PlanReActPlanner()` forces non-thinking or third-party models to emit structured `/*PLANNING*/` → `/*ACTION*/` → `/*REASONING*/` → `/*FINAL_ANSWER*/` tags.

Reasoning pattern	Flow	Choose when
ReAct	Think → act → observe, repeat	Exploratory work; step N+1 depends on step N
ReWOO	Plan once → run tools in parallel → solve once	Independent sub-tasks known upfront (~50–70% fewer tokens than ReAct)
LLMCompiler	Planner emits a task DAG → dispatcher runs ready tasks in parallel → joiner	Interdependent tasks with parallel branches
Parallel fan-out	`ParallelAgent` or `Workflow` tuple + `Join Node`	Isolated multi-source queries

6. State, sessions and memory

Store	What it holds	Lifetime	Accessed via
Session events	Transcript of one conversation thread	The session	`SessionService`
State	Exact key/value working data	Depends on prefix (below)	`output_key`, `state_delta`, `tool_context.state`
Memory	LLM-extracted, consolidated facts across sessions, retrieved by similarity	Long-term, per user	`MemoryService` + `load_memory` / `PreloadMemoryTool`
Artifacts	Files and binary outputs (versioned; `"user:..."` filename = cross-session)	Session or user	`ArtifactService`, `tool_context.save_artifact`

State key prefix	Scope	Persistence
(none)	This session	With the session (requires `Database SessionService` or `VertexAi SessionService`)
`user:`	One user across all sessions of the same `app_name`	Persistent **only** with `Database SessionService` / `VertexAi SessionService`
`app:`	All users and sessions of the `app_name`	Persistent **only** with `Database SessionService` / `VertexAi SessionService`
`temp:`	Current invocation only (shared by sub-agents in the same turn)	**Never persisted** (stripped before storage)

- **Write state through the event system:** `output_key`, `Event Actions(state_delta={...})`, or `tool_context.state` / `callback_context.state` (tracked into a delta automatically).
- **Memory Bank vs. RAG Memory vs. Grounding:**
- **`VertexAiMemoryBankService`**: LLM-extracted, **consolidated** user facts that update when preferences change. `await memory_service.add_session_to_memory(session)` runs `GenerateMemories` (extraction + consolidation, best wired in `after_agent_callback`); `add_memory(...)` calls `CreateMemory` (writes verbatim facts without consolidation unless `enable_consolidation=True`).
- **`VertexAiRagMemoryService`**: stores raw session transcripts in a Vertex AI RAG corpus (`similarity_top_k`, `vector_distance_threshold` where **lower = closer**); does **not** consolidate contradictions.
- **Document grounding tools** (shared knowledge base, not user memory): `google_search` (public web), `VertexAiSearchTool` (Vertex AI Search data store/engine) and `VertexAiRagRetrieval` (Vertex AI RAG Engine corpus).

Read tool	Behavior	Trade-off
`load_memory`	The model decides when to search	Cheaper; misses recall if the model doesn't think to search
`PreloadMemoryTool`	Retrieves memories at the start of every turn	Always grounded; adds latency and tokens every turn

> **COMMON TRAP**
> editing `session.state` on an object returned by `get_session()` bypasses the event ledger: no history, does not persist to DB/Vertex, race-prone. Treat it as read-only. And `user:` / `app:` keys still vanish on restart if you leave `InMemorySessionService` configured.

7. Tools and structured output

`tools=[my_function]` auto-wraps a Python callable as a `FunctionTool`: the **docstring** becomes the description and **type hints** become the JSON schema. The model selects and fills tools from that text alone, so **docstring quality is a production safeguard.**

Rule	Why	Example
Atomic: one tool, one job	A `research_topic` tool hides an agent inside a tool	`web_search` + `web_read_page`
Domain-prefixed names	Avoids collisions, helps selection	`file_read`, `db_query`
State limits in the docstring	The model plans around known constraints	"Read-only; max 1 MB; 30 s timeout."
Return errors, don't raise	A structured error lets the model retry or escalate; an uncaught exception ends the turn	`{"status": "error", "error_message": "Not found: a.csv"}`
Allowlist inside the tool	Safety must not depend on the prompt or brittle keyword blocklists	`resolve()` + `is_relative_to(base)`; read-only DB role / `WriteMode.BLOCKED`
Idempotent side effects	Retries and resumption are at-least-once	Check-before-create, idempotency keys
Keep the catalogue small	Tool selection degrades past roughly **16 tools** per agent	Split into sub-agents or skills

```
def file_read(path: str) -> dict:
    """Read a UTF-8 text file inside the project sandbox (read-only,
max 1 MB).

    Args:
        path: Path relative to the project root.
    """
    target = (BASE_DIR / path).resolve()
    if not target.is_relative_to(BASE_DIR):
        return {"status": "error", "error_message": "Path outside
sandbox."}
    return {"status": "success", "content": target.read_text()
[:1_000_000]}
```

- **Tool families:** `FunctionTool` (`async def` functions run concurrently when the model emits parallel tool calls), `AgentTool`, `LongRunningFunctionTool`, Gemini built-ins (`google_search`, `VertexAiSearchTool`, `BuiltInCodeExecutor`), Google Cloud toolsets (`BigQueryToolset` with `BigQueryToolConfig(write_mode=WriteMode.BLOCKED)`, `SpannerToolset`, `ApiHubToolset`, `ApplicationIntegrationToolset`, `ToolboxToolset`), `OpenAPIToolset`, `McpToolset`, `SkillToolset`.
- **Gemini built-in tool constraint:** built-in tools (`google_search`, `VertexAiSearchTool`, `BuiltInCodeExecutor`) execute inside the Gemini API call and cannot be freely mixed with custom function tools or placed inside `sub_agents` transfer targets on all models. Either set `bypass_multi_tools_limit=True` on Python search tools (`GoogleSearchTool(bypass_multi_tools_limit=True)`, `VertexAiSearchTool(..., bypass_multi_tools_limit=True)`) or isolate each built-in on its own specialist wrapped in `AgentTool(agent=...)`.
- **Structured output:** `output_schema=<Pydantic model>` forces the agent's final answer into a schema; combine with `output_key` to pass typed data to the next agent. For high-stakes pipelines add epistemic fields (`confidence`, `citations`, `escalation_required: bool = True` by default, so the pipeline must *earn* auto-approval).

> **COMMON TRAP**
> on models/SDK versions where `output_schema` disables tool calling (or forces an extra schema-conversion step), let a tool-using agent gather data and add a separate formatter agent (`output_schema=MySchema`, `include_contents='none'`) at the end of a `SequentialAgent`.

8. MCP and A2A integration

Vertical (agent → tools/data) = MCP. Horizontal (agent → agent) = A2A. They complement each other; see the MCP and A2A guides for protocol detail.

```
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import
StdioConnectionParams, StreamableHTTPConnectionParams
from mcp import StdioServerParameters

local_fs = McpToolset(

connection_params=StdioConnectionParams(server_params=StdioServerParame
ters(
        command="npx", args=["-y", "@modelcontextprotocol/server-
filesystem", "/data"])),
    tool_filter=["read_file", "list_directory"],     # least privilege:
names list or a predicate
    tool_name_prefix="fs",                           # prevents
collisions across multiple MCP servers
)
remote_rules =
McpToolset(connection_params=StreamableHTTPConnectionParams(url="https:
//rules.internal/mcp"))
```

MCP topic	Rule
Transports	**stdio** = local subprocess (dev/desktop); **Streamable HTTP** = remote/shared (production); SSE-only is legacy
Least privilege	`tool_filter` takes an explicit **list of tool names** or a **predicate** (`ReadonlyContext` aware), not glob patterns; `require_confirmation` gates sensitive MCP tools
Resources & Expose	`use_mcp_resources=True` adds `list_mcp_resources` / `read_mcp_resource` tools; expose ADK tools as an MCP server via `to_mcp_server(tool)` or `adk_to_mcp_tool_type(tool)`
Managed GCP MCP	`ApiRegistry(mcp_server_name=...)` (Cloud API Registry), `ToolboxToolset(server_url=...)` (MCP Toolbox for Databases with pooled connections), `AgentRegistry` (discover MCP servers and A2A agents in one catalog)
Lifecycle	Outside `adk web`, `await toolset.close()` on shutdown; define toolsets **synchronously** at module level in `agent.py` for deployment

A2A topic	Rule
Consume	`RemoteA2aAgent(name=..., description=..., agent_card="https://host/.well-known/agent-card.json")`
Expose	`to_a2a(agent, port=8001)` (serve with uvicorn) or `adk api_server --a2a`
Transport	JSON-RPC 2.0 / gRPC / HTTP+JSON over HTTPS, SSE streaming (`message/stream`), push notifications (webhooks) for long tasks
Task lifecycle	`submitted` → `working` → `input-required` / `auth-required` → `completed` / `failed` / `canceled` / `rejected`
Trust	Remote agents run **outside** your callbacks and guardrails: authenticate via the card's `securitySchemes` and validate every returned artifact

Protocol	Connects	Key safety idea
MCP	Agent → tools and data	Treat tools like public APIs; `tool_filter` ; read-only by default
A2A	Agent → agent	Opaque peers; validate artifacts
A2UI	Agent → UI	Client renders only components from its trusted catalogue
AP2	Agent → payments (A2A extension)	Signed mandates (Intent → Cart → Payment): no mandate, no payment

> **COMMON TRAP**
> A2A does **not** expose a remote agent's tools, prompts or memory, only its card (`/.well-known/agent-card.json`), tasks and artifacts. If you need its tools directly, you need an MCP server.

9. Skills and progressive disclosure

A **tool** is a function; a **skill** is packaged expertise (instructions + resources + scripts) loaded **only when needed**, so the prompt doesn't inflate as capabilities grow.

Tier	Content	Loaded by
L1 metadata	YAML frontmatter `name` (kebab-case matching directory) + `description` (~60 tokens)	`list_skills` (or injected eagerly into system prompt)
L2 instructions	Full `SKILL.md` body (< 500 lines)	`load_skill`
L3 resources	`references/` , `assets/` , `scripts/`	`load_skill_resource` , `run_skill_script` (requires a `code_executor` on the agent/toolset)

- **ADK wiring:** `load_skill_from_dir(Path("./skills/my-skill"))` or inline `models.Skill(frontmatter=..., instructions=...)` passed to `SkillToolset(skills=[...], code_executor=...)` ; with a `Skill Registry` (e.g. `GCPSkillRegistry`) the agent also gets `search_skills` .
- **Authoring & bundled scripts:** the frontmatter `description` is the routing index, so write it the way users phrase requests. Scripts must be **(1) non-interactive** (CLI flags/env vars only; stdin hangs in headless containers), **(2) self-documenting** (`--help`), and **(3) emit structured JSON on stdout** with actionable errors on stderr + non-zero exit codes.
- **Toolset ≠ skill:** a group of related tools sharing config (e.g. `Mcp Toolset` , any `BaseToolset`) is a **toolset**, not a `SKILL.md` skill.

> **COMMON TRAP**
> a malicious `SKILL.md` or script is prompt injection through the filesystem. Load skills only from allowlisted roots, review them like code and provide a sandboxed `code_executor` when enabling `run_skill_script` .

10. Callbacks, plugins and guardrails

Harness engineering: anything that must *always* happen (validation, limits, redaction, approval) belongs in deterministic code around the model, not in instructions the model may skip.

Hook	Return `None`	Return a value
`before_agent_callback(callback_context)`	Agent runs	`types.Content` → agent run **skipped**; content is the reply
`after_agent_callback(callback_context)`	Output unchanged	`types.Content` → **appended** as an extra event (does not replace the output)
`before_model_callback(callback_context, llm_request)`	Model called (request may be edited in place)	`LlmResponse` → model call **skipped** (guardrail, cache)
`after_model_callback(callback_context, llm_response)`	Response unchanged	`LlmResponse` → **replaces** the response (output backstop, disclaimers)
`on_model_error_callback(callback_context, llm_request, error)`	Exception re-raised	`LlmResponse` → exception **suppressed**; response used (`after_model_callback` still runs)
`before_tool_callback(tool, args, tool_context)`	Tool runs (`args` dict may be mutated in place)	`dict` → tool **skipped**; dict is the result (policy block, rate limit, cache, mock)
`after_tool_callback(tool, args, tool_context, tool_response)`	Result unchanged	`dict` → **replaces** the tool result (PII scrubbing, indirect-injection stripping)
`on_tool_error_callback(tool, args, tool_context, error)`	Exception re-raised	`dict` → exception **suppressed**; dict is the fallback result (`after_tool_callback` still runs)

Guardrail layer	Check (cheapest first)	Hook
Input	Regex injection patterns (~1 ms) → small classifier model for scope/jailbreak	`before_model_callback` (or `before_agent_callback`)
Pre-LLM reality check	Wrong file type, blank scan, low parse quality → escalate without spending tokens	`before_agent_callback`
Tool arguments	Path traversal, SQL/shell injection, amount limits, PII before external calls	`before_tool_callback`
Tool results	Indirect injection in pages/emails/PDFs; **redact PII before it enters model context & session events**	`after_tool_callback`
Output	Final PII backstop, groundedness/citation check	`after_model_callback`

Guardrail layer	Check (cheapest first)	Hook
Platform	Gemini `safety_settings`, Vertex AI **Model Armor** prompt/response/tool screening	`GenerateContentConfig` / `BasePlugin`

```
def block_injection(callback_context, llm_request):
    last = llm_request.contents[-1] if llm_request.contents else None
    text = (last.parts[0].text or "") if last and last.parts else ""
    if INJECTION_PATTERN.search(text):
        return LlmResponse(content=types.Content(role="model",
                           parts=[types.Part(text="I can't help with
that request.")]))
    return None                                  # proceed to the
model

agent = LlmAgent(name="assistant", model="gemini-3.8-flash",
before_model_callback=block_injection)
```

- **Plugins (`BasePlugin`):** register once on `App(name=..., root_agent=..., plugins=[...])` (`Runner(plugins=...)` is deprecated) to apply hooks across **every** agent, model and tool call in the hierarchy. **Execution order:** plugin callbacks run **before** agent-level callbacks, and the first non-`None` return short-circuits remaining hooks in that phase.
- **Escalation thresholds:** map a composite quality score to deterministic actions, e.g. ≥ 0.85 proceed, 0.70–0.85 flag for verification, < 0.70 hard stop and human escalation.

> **COMMON TRAP**
> redacting PII only in `after_model_callback` is too late: raw tool outputs have already been sent to the LLM and persisted in `session.events`. Scrub PII at the **tool boundary** (`after_tool_callback` and `before_tool_callback`), and keep `after_model_callback` as the final output backstop.

11. Human-in-the-loop, resilience and durability

> **RULE OF THUMB**
> if a step can change user data, send something externally or spend money, pause for approval before it runs.

Need	ADK mechanism
Yes/no approval before a tool runs	`FunctionTool(fn, require_confirmation=True)`
Threshold approval (e.g. only when `amount > 1000`)	`FunctionTool(fn, require_confirmation=lambda amount, **_: amount > 1000)` (also on `McpToolset`)
Structured approval with custom human payload	`tool_context.request_confirmation(hint=..., payload={...})` inside the tool; read `tool_context.tool_confirmation.payload` on resume
Pause a graph node for human input (ADK 2.0)	Yield `RequestInput(message=..., schema=...)` inside a generator node
Job that finishes minutes/hours later (ticket, webhook)	`LongRunningFunctionTool`: returns a pending dict now; client later sends the final `FunctionResponse` with the matching `function_call.id`
Resume an interrupted invocation	`App(..., resumability_config=ResumabilityConfig(is_resumable=True))` + `runner.run_async(..., invocation_id=...)` (**at-least-once**)
Cap runaway runs	`RunConfig(max_llm_calls=...)` (default **500** per run), `LoopAgent(max_iterations=...)`

Exception handling ladder across ADK's 3 recovery layers: 1. **Transport (HTTP 429/503):** configure `HttpRetryOptions(attempts=3, initial_delay=1.0, exp_base=2.0)` on the `Gemini` model's `retry_options`. 2. **Tool self-correction:** `ReflectAndRetryToolPlugin(max_retries=3, throw_exception_if_retry_exceeded=False)` feeds tool errors back to the LLM so it can fix malformed arguments. **Caveat:** by default it only intercepts **raised** tool exceptions; if your tools return `{"status": "error", ...}` dicts, override `extract_error_from_result` on the plugin. 3. **Model failover & human escalation:** catch persistent model/quota errors in `on_model_error_callback` (Python) or `RoutedLlm` (TypeScript, experimental) to switch models, and escalate to a human after N consecutive tool failures tracked in `state`.

- **Idempotency:** retries and resumption can re-execute a completed tool call if the process crashed before the event commit, so side-effecting tools need check-before-create or idempotency keys.
- **Goal monitoring:** store explicit success criteria in state and let a checker agent or callback compare progress to them; escalate, don't loop, when the goal is unreachable.

12. Context engineering and performance

A large context window does not prevent drift: a specialist that inherits a long, noisy trajectory performs worse. **Pass curated summaries, not transcripts.**

Knob	Where	Effect	Use when
`static_instruction` vs. `instruction`	`LlmAgent`	`static_instruction` stays cache-stable; dynamic `{state_key}` placeholders in `instruction` append after it	Avoiding prompt-cache invalidation on every turn
`include_contents='none'`	`LlmAgent`	Agent sees only its instruction and current input	Stateless specialists (classifiers, formatters, auditors)
`output_key` + `{key}` templating	`LlmAgent`	Pass a curated result through state	Pipeline steps needing only the previous output
`AgentTool`	Parent's `tools`	Child's intermediate tool calls stay out of the parent's message history	Exploratory sub-tasks
`EventsCompactionConfig(compaction_interval, overlap_size)`	`App`	Summarizes older event ranges via an LLM into a compacted event	Long-running sessions
`ContextCacheConfig(cache_intervals, ttl_seconds, min_tokens)`	`App`	Reuses cached static prefixes on Gemini 2.0+	Long instructions / tool schemas called repeatedly
Skills (`SkillToolset`)	`tools`	Loads instructions on demand	Large capability catalogues

Resource-aware routing: use eval results (task success, `tool_trajectory_avg_score`) per task type to route easy requests to a cheaper model (`gemini-3.8-flash` / Flash-Lite) and reserve Pro + high `thinking_budget` for hard ones.

COMMON TRAP

`EventsCompactionConfig`, `ResumabilityConfig` and `ContextCacheConfig` are **`App`-level** settings, not `LlmAgent` parameters. And putting dynamic `{timestamp}` or `{user_status}` inside `static_instruction` (or at the top of a single prompt without `static_instruction`) busts the prefix cache on every call.

13. Security and sandboxing

Threat	Example	ADK mitigation
Direct prompt injection	"Ignore previous instructions..."	`before_model_callback` (regex → classifier), Model Armor
Indirect injection (dominant agent threat)	Hidden text in a fetched page, email or PDF	Scan every external payload in `after_tool_callback`; isolate untrusted reader from action tools
Confused deputy / excessive agency	A summarizer with `send_email` is tricked into using it	Remove capabilities at the registry: `tool_filter`, per-agent identity, read-only credentials
Cascading injection	Tainted researcher output flows to writer → publisher	Treat every handoff as untrusted: `output_schema`, callback checks between steps
Agent impersonation	Forged Agent Card	Authenticated endpoints (mTLS/OAuth), signed cards
Runaway cost	Loops or token-bomb inputs	`max_llm_calls`, `max_iterations`, per-session budget in state
Supply chain	Malicious skill or hallucinated package name	Allowlisted skill roots, code review, pinned `npx` / `uvx` versions

Code executor (`LlmAgent(code_executor=...)`)	Where code runs	Use when
`BuiltInCodeExecutor`	Gemini's server-side sandbox (stateless, 30 s limit, no internet)	Quick math, charts and data analysis; zero infra
`AgentEngineSandboxCodeExecutor`	Managed Agent Runtime sandbox (stateful across turns, up to 14 days TTL)	Production on Google Cloud
`GkeCodeExecutor`	GKE **Agent Sandbox** on gVisor (`executor_type="sandbox"`, pre-warmed; legacy `"job"` mode spins a K8s Job per run)	GKE-hosted, kernel-level isolation with sub-second startup
`ContainerCodeExecutor`	Self-hosted Docker container	Custom dependencies in staging/self-hosted setups
`UnsafeLocalCodeExecutor`	Host process (`exec()`, full host access, stateless)	Local experiments only, **never** production

- **Three identity models for tool access: (1) Agent Identity** (recommended on Agent Runtime: each agent gets its own Google Cloud IAM principal `principal://...`, independent of any service account, for least-privilege access and per-agent audit logs), **(2) Service account** (workload identity for shared infra), and **(3) OAuth 2.0 user delegation** (`ToolContext.request_credential(auth_config)` when acting on a specific user's behalf).
- **Zero-trust checklist:** least privilege per agent · validate at every boundary (`before_tool`, `after_tool`, `output_schema`) · audit every call (event ledger + OpenTelemetry) · rate-limit risky actions (counter in state) · human gate for irreversible actions.

14. Evaluation and observability

Grade the path, not just the answer. An agent that answers correctly after 15 erratic tool calls (where 3 would do) is still failing in production.

Artifact	Purpose	When
`*.test.json`	Unit-style: one session, simple interactions	Every commit
`*.evalset.json`	Integration-style: multiple, multi-turn sessions (`EvalSet` / `EvalCase`)	Pre-release
`test_config.json`	Criteria thresholds + `custom_metrics` (`code_config.name`)	With each run

Four ways to run: `adk web` (Eval tab: save live sessions as cases, inspect traces), `adk eval <agent_dir> <evalset.json>` (CLI/CI), `await AgentEvaluator.evaluate(...)` in `pytest`, and **`adk conformance test`** (`--mode=replay` verifies deterministic harness wiring without live model variance; `--mode=live` checks live drift against `adk conformance record` baselines).

Criterion	Measures	Judge
`tool_trajectory_avg_score`	Actual vs. expected tool calls. `match_type`: `EXACT` (default, no extra or missing calls), `IN_ORDER` (extras allowed between), `ANY_ORDER`	Deterministic
`response_match_score`	ROUGE-1 word overlap with the reference	Deterministic
`final_response_match_v2`	Semantic equivalence to the reference	LLM
`rubric_based_final_response_quality_v1` / `rubric_based_tool_use_quality_v1`	Custom rubrics	LLM
`hallucinations_v1`	Groundedness in context (catches ungrounded numbers even when the right tool ran)	LLM
`safety_v1`	Harmlessness (delegates to Vertex AI Eval Service)	LLM
`multi_turn_task_success_v1` / `multi_turn_trajectory_quality_v1`	Whole-conversation success with `UserSimulatorProvider` (`conversation_scenario` + `starting_prompt` + `user_persona`)	LLM

- **Built-in efficiency metrics vs. custom thresholds:** ADK automatically emits `tool_call_count_v1`, `inference_call_count_v1`, `token_usage_v1` and `invocation_duration_v1` as **`INFORMATIONAL`** metrics (setting a pass/fail threshold on them in `test_config.json` raises an error). To fail a test on call budgets or forbidden tools, register a Python function under `custom_metrics` (`code_config: {"name": "pkg.mod.metric_fn"}`).
- **LLM-as-judge rules:** grade against reference facts, require **reasoning before the score**, return JSON; a 3-judge majority vote stabilizes borderline cases. Keep red-team cases (jailbreaks, indirect injection, empty/huge inputs) in the evalset permanently.

- **Observability:** native OpenTelemetry spans (`invoke_agent`, `call_llm`, `execute_tool`); `--otel_to_cloud` exports to Cloud Trace (`--trace_to_cloud` is deprecated); set `ADK_CAPTURE_MESSAGE_CONTENT_IN_SPANS=false` in production so raw prompts, tool args and PII are omitted from traces.

> **COMMON TRAP**
> `response_match_score` is **lexical** (ROUGE-1), so a correct paraphrase can fail. Use `final_response_match_v2` or a rubric when wording legitimately varies. And `EXACT` trajectory matching fails agents that make harmless extra calls; use `IN_ORDER` or `ANY_ORDER` when order or extras don't matter.

15. Deployment, live streaming and developer tooling

The managed-agent split: the platform owns the loop, sandbox, sessions, scaling and restarts; **you own** instructions, tools and their permissions, and what the agent outputs.

Target	Command	You manage	Choose when
Agent Runtime (formerly Agent Engine)	`adk deploy agent_engine --project --region --display_name <dir>`	Agent code only	Default on Google Cloud: managed sessions, Memory Bank, code sandbox, Agent Identity, Gemini Enterprise publishing
Cloud Run	`adk deploy cloud_run --project --region [--trigger_sources="pubsub,eventarc"] <dir>`	Container, scaling, session DB	Custom FastAPI routes, scale-to-zero, **ambient (event-driven) agents** triggered by Pub/Sub or Eventarc
GKE	`adk deploy gke ... <dir>`	Cluster, networking, autoscaling	Network/residency control, GPU co-location, gVisor Agent Sandbox
Self-hosted	`adk api_server` or `get_fast_api_app(...)`	Everything	On-prem, other clouds, integration tests

Command	Purpose
`adk run <dir>`	Terminal chat with the agent
`adk web`	Browser dev UI: chat, event trace, state, Eval tab
`adk eval <dir> <evalset>`	Run evals in CI
`adk conformance record` / `adk conformance test`	Record golden trajectories and run deterministic replay or live conformance tests
`adk api_server [--a2a] [--otel_to_cloud]`	Local FastAPI server (optionally A2A endpoints and Cloud Trace export)
`agents-cli deploy` / `agents-cli eval run`	Project lifecycle CLI: deploys to the `deployment_target` in `pyproject.toml` and runs eval suites

```
import asyncio
from google.genai import types
from google.adk.agents import Agent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

agent = Agent(name="assistant", model="gemini-3.8-flash",
instruction="You are a helpful assistant.")

async def main(query: str) -> str:
    session_service = InMemorySessionService()            # swap for
Database/VertexAi in prod
    session = await session_service.create_session(app_name="demo",
user_id="u1")
    runner = Runner(agent=agent, app_name="demo",
session_service=session_service)
    msg = types.Content(role="user", parts=[types.Part(text=query)])
    final_text = ""
    async for event in runner.run_async(user_id="u1",
session_id=session.id, new_message=msg):
        if event.is_final_response() and event.content and
event.content.parts:
            # In multi-agent workflows EVERY sub-agent emits
is_final_response(); drain the stream
            # and skip thinking parts (p.thought == True) so you keep
the last agent's answer.
            final_text = "".join(p.text or "" for p in
event.content.parts if not getattr(p, "thought", False))
    return final_text

print(asyncio.run(main("Hello!")))
```

- **Live (bidirectional audio/video) streaming:** use `runner.run_live(user_id=..., session_id=..., live_request_queue=LiveRequestQueue(), run_config=RunConfig(response_modalities=["AUDIO"]))` (`send_realtime(Blob)` for audio/video frames, `send_content(Content)` for text, `close()` on exit) and stop playback when `event.interrupted == True`. In Python ADK, setting `StreamingMode.BIDI` on `run_async()` is a silent no-op: only `run_live()` opens a Gemini Live session (`StreamingMode.SSE` on `run_async()` streams text tokens).
- On Agent Runtime the resource name is `projects/.../reasoningEngines/<id>`; managed sessions use `--session_service_uri agentengine://<id>`.

16. Failure modes and exam traps

Symptom / scenario	Root cause → correct fix	Tempting wrong answer
Conversation history or `user:` state lost after a restart	`InMemorySessionService` in production → `DatabaseSessionService` / `VertexAiSessionService`	Switching to `InMemoryMemoryService`
Users see each other's data under load	Memory stored on the agent object → stateless `Agent`; state in `SessionService` keyed by app/user/session	One agent instance per user
`LoopAgent` always runs every iteration	Nothing sets `escalate` → exit tool sets `tool_context.actions.escalate = True` (`skip_summarization = True`)	Raising an exception in the tool
Next step runs even when `LoopAgent` never converged	`LoopAgent` exits **silently** at `max_iterations` → set `state["temp:approved"] = True` in exit tool and check it in `before_agent_callback`	Expecting `LoopAgent` to raise `TimeoutError`
`ParallelAgent` results overwrite each other	Branches share one `output_key` → distinct key per branch, then a synthesizer (or `JoinNode` in ADK 2.0)	Switching to `SequentialAgent`

Symptom / scenario	Root cause → correct fix	Tempting wrong answer
Specialist takes over the chat when you wanted a sub-result	`sub_agents` transfer → wrap the child in `AgentTool` (or ADK 2.0 `mode="single_turn"`)	Better instructions for the child
`KeyError` when instruction includes a JSON example `{"id": 1}`	ADK treats `{id}` as a state placeholder → wrap the prompt in an `InstructionProvider` callable `def prompt(ctx):`	Escaping with double braces `{{id}}`
Early `return` on `event.is_final_response()` returns step 1's output	In multi-agent workflows **every** sub-agent emits `is_final_response()` → drain the `run_async` generator (and filter `not p.thought`)	Parsing `session.events` manually
Combining `google_search` with custom `FunctionTool`s errors	Gemini built-ins execute server-side → set `bypass_multi_tools_limit=True` or wrap the search agent in `AgentTool`	Switching to `SequentialAgent`
`ReflectAndRetryToolPlugin` never retries a failing tool	Tool returns `{"status": "error"}` instead of raising → override `extract_error_from_result` on the plugin	Raising `max_retries` to 10
Tool confirmation crashes after moving to `DatabaseSessionService`	Built-in tool confirmation is `InMemorySessionService`-only → use `LongRunningFunctionTool` / external approval workflow	Adding an async DB driver
Agent runs a destructive MCP or DB tool	All tools/verbs exposed → `tool_filter=[...]` names list, `WriteMode.BLOCKED`, read-only DB role	"You are read-only" in the prompt or `DROP` blocklist

Symptom / scenario	Root cause → correct fix	Tempting wrong answer
Hidden instruction in a fetched PDF hijacks the agent	Only user input screened → `after_tool_callback` payload scanning + isolate reader from action tools	Input guardrail only
PII leaks into LLM context and `session.events`	Redaction only in `after_model_callback` → scrub PII at the tool boundary (`after_tool_callback` & `before_tool_callback`)	"Never reveal PII" instruction
`after_agent_callback` didn't change the reply	It **appends** an event → replace model output in `after_model_callback`	Returning a longer `Content`
Approved payment executed twice after a resume	Resumability is at-least-once → idempotency keys, check-before-create	Disabling resumability
Whole agent team down during a model outage	Same hardcoded model everywhere → `HttpRetryOptions` for 429/503 + `on_model_error_callback` model failover	Retrying the same model forever
Correct agent fails trajectory eval or paraphrased response eval	`EXACT` trajectory / lexical `response_match_score` (ROUGE-1) → use `IN_ORDER` / `ANY_ORDER` and `final_response_match_v2`	Lowering the threshold to 0.1
Setting a threshold on `tool_call_count_v1` errors in `adk eval`	Built-in efficiency metrics are `INFORMATIONAL` only → define a `custom_metrics` function in `test_config.json`	Switching to `adk web`
Prompt cache hit rate is 0% despite `ContextCacheConfig`	Dynamic `{state_key}` placed at top of prompt → put static text in `static_instruction` and dynamic state in `instruction`	Increasing `ttl_seconds`
Deployed agent fails to start with MCP tools	Async toolset factory → define `McpToolset` synchronously at module level in `agent.py`	Increasing container memory